Part A – NumPy

Question 1

Create a NumPy array containing the integers from 1 to 20.

Perform the following operations:

1. Display the array.
2. Find the shape of the array.
3. Find the maximum and minimum values.
4. Calculate the mean and standard deviation.
5. Reshape the array into a 4 × 5 matrix.
6. Display the second row of the matrix.
7. Display the third column of the matrix.

Question 2

Create two NumPy arrays:

A = [10, 20, 30, 40, 50]

B = [5, 10, 15, 20, 25]

Perform:

1. Addition of A and B.
2. Subtraction of B from A.
3. Element-wise multiplication.
4. Element-wise division.
5. Find the sum of all elements in A.

Part B – Pandas

Question 3

Create a DataFrame using the following student data:

Roll No	Name	Marks
101	Ravi	85
102	Priya	92
103	Kiran	78
104	Anu	88
105	Sita	95

Perform the following operations:

1. Display the DataFrame.
2. Display the first three records.
3. Find the average marks.
4. Find the student with the highest marks.
5. Find the student with the lowest marks.
6. Add a new column called "Grade" using:
    - A for Marks ≥ 90
    - B for Marks between 80 and 89
    - C for Marks < 80

Question 4

Create a DataFrame containing the following information:

Product	Price	Quantity
Laptop	50000	10
Mouse	500	50
Keyboard	1200	30
Monitor	10000	15

Perform:

1. Calculate the total value of each product using:

    Total Value = Price × Quantity

2. Add the Total Value column.
3. Find the product with the highest total value.
4. Calculate the overall inventory value.

Part C – Dataset Exploration

Question 5

Download the Iris dataset from Scikit-learn and perform the following:

1. Load the dataset into a Pandas DataFrame.
2. Display the first five records.
3. Find the number of rows and columns.
4. Display column names.
5. Check for missing values.
6. Generate summary statistics using describe().
7. Find the number of samples belonging to each class.

Aim

To perform exploratory data analysis (EDA) on a student performance dataset using Pandas and Matplotlib.

Dataset

Create a CSV file named student_data.csv with the following data:

RollNo	Name	Gender	Attendance	InternalMarks	ExternalMarks
101	Ravi	M	85	32	55
102	Priya	F	92	38	58
103	Kiran	M	78	28	45
104	Anu	F	95	40	60
105	Sita	F	88	35	54
106	Raj	M	70	25	40
107	Divya	F	90	37	57
108	Arun	M	82	30	48
109	Keerthi	F	96	39	59
110	Naveen	M	75	27	44

Tasks

Task A: Dataset Loading and Inspection

- Import the necessary Python libraries.
- Load the dataset into a Pandas DataFrame.
- Display the first five records.
- Display the last five records.
- Find the total number of rows and columns.
- Display the names of all columns.
- Display the data type of each column.
- Generate dataset information using info().

Task B: Statistical Analysis

- Calculate the mean attendance of all students.
- Calculate the average Internal Marks.
- Calculate the average External Marks.
- Find the maximum Internal Marks.
- Find the minimum Internal Marks.
- Find the maximum External Marks.

- Find the minimum External Marks.
- Calculate the standard deviation of Internal Marks.
- Generate summary statistics using describe().

Task C: Data Manipulation

- Create a new column named TotalMarks**.**
- TotalMarks=InternalMarks+ExternalMarks
- Create a new column named Percentage**.**
    1. Percentage=TotalMarks/100×100
- Display the updated DataFrame.

Task D: Filtering and Searching

- Display the details of students whose attendance is greater than 90%.
- Display the details of students whose TotalMarks exceed 90.
- Display the details of female students.
- Display the details of male students.
- Display students whose ExternalMarks are less than 50.

Task E: Ranking and Comparison

- Identify the student with the highest TotalMarks.
- Identify the student with the lowest TotalMarks.
- Display the top three performers based on TotalMarks.
- Display the bottom three performers based on TotalMarks.
- Determine how many students scored above the class average.

Task F: Group-wise Analysis

- Calculate the average TotalMarks of male students.
- Calculate the average TotalMarks of female students.
- Compare the average attendance of male and female students.

Task G: Data Visualization

- Plot a bar chart showing TotalMarks of all students.
- Plot a histogram of Attendance.
- Plot a pie chart showing the distribution of male and female students.
- Plot a scatter plot between Attendance and TotalMarks.
- Interpret the relationship between Attendance and TotalMarks.

Task H: Observations

- Write any three observations based on the analysis performed.
- State whether higher attendance appears to be associated with better academic performance.

Implement the ID3, C4.5, and CART decision tree algorithms from scratch (without using any machine learning libraries) for a suitable classification dataset. Develop the complete training procedure by implementing the required attribute selection measures, construct the decision trees, and use the generated models to classify test instances. Compare the three algorithms in terms of classification accuracy, precision, recall, F1-score, tree depth, number of leaf nodes, and training time**,** and summarize your observations on their performance and the resulting tree structures.

Task 1: Hidden Markov Model for Human Activity Recognition

Problem Statement

A wearable fitness tracker is used to monitor a person's physical activity based on sensor readings. The actual activity being performed is *hidden*, while the wearable records the intensity of movement during fixed time intervals. The movement intensity is categorized as *Low*, *Medium*, or *High*.

The Hidden Markov Model (HMM) is defined as follows.

- **Hidden States:**

$$S = \{\text{Walking},\ \text{Running},\ \text{Resting}\}$$

- **Observation Symbols:**

$$V = \{\text{Low},\ \text{Medium},\ \text{High}\}$$

- **Initial State Probability Vector:**

$$\pi = \begin{bmatrix} 0.5 & 0.3 & 0.2 \end{bmatrix}$$

- **State Transition Probability Matrix:**

$$A = \begin{bmatrix} 0.6 & 0.3 & 0.1 \\ 0.2 & 0.5 & 0.3 \\ 0.3 & 0.2 & 0.5 \end{bmatrix}$$

  where each row represents the current hidden state and each column represents the next hidden state.

- **Emission Probability Matrix:**

$$B = \begin{bmatrix} 0.6 & 0.3 & 0.1 \\ 0.1 & 0.3 & 0.6 \\ 0.7 & 0.2 & 0.1 \end{bmatrix}$$

  where each row represents a hidden state (Walking, Running, Resting) and each column represents an observation symbol (Low, Medium, High).

- **Observation Sequence:**

$$O = (\text{High},\ \text{Medium},\ \text{Low},\ \text{High})$$

Tasks

1. Compute the forward probabilities for each time step using the Forward Algorithm.
2. Construct the complete Forward Probability (α) table.
3. Determine the likelihood of the given observation sequence.
4. Implement the Forward Algorithm in Python and verify the computed likelihood.

Task 2: Human Activity Recognition using `hmmlearn`

Problem Statement

Using the Hidden Markov Model parameters provided in Task 1, implement the HMM using the `hmmlearn` Python library.

Tasks

1. Install the `hmmlearn` library and import the required modules.
2. Construct a Categorical Hidden Markov Model using the given initial state probabilities, transition probability matrix, and emission probability matrix.
3. Compute the likelihood of the observation sequence

$$O = (\text{High},\ \text{Medium},\ \text{Low},\ \text{High}).$$

4. Compute the llikelihood for each of the following observation sequences:
   - $O_1 = (\text{Low},\ \text{Medium},\ \text{High},\ \text{Medium})$
   - $O_2 = (\text{High},\ \text{High},\ \text{High},\ \text{High})$
   - $O_3 = (\text{Low},\ \text{Low},\ \text{Low},\ \text{Low})$
   - $O_4 = (\text{Medium},\ \text{Medium},\ \text{Medium},\ \text{Medium})$
5. Briefly explain the relationship between the observation likelihood computed using the Forward Algorithm and the likelihood obtained using `hmmlearn`.

Experiment: Simple Linear Regression

Experiment Title

Implementation and Evaluation of Simple Linear Regression

Objective

The objective of this experiment is to implement a Simple Linear Regression model to predict a continuous target variable, visualize the best-fit regression line, and evaluate the model using standard regression performance metrics.

Problem Statement

House prices generally increase with an increase in the size of the house. In this experiment, you will develop a Simple Linear Regression model to predict the price of a house based on its size.

Use the following dataset to build, train, and evaluate the regression model.

Table 1: House Price Dataset

House Size (sq.ft.)	**House Price (Lakhs)**
600	25
800	32
1000	38
1200	45
1400	52
1600	58
1800	65
2000	72
2200	80
2400	87

Tasks

1. Create the dataset using Pandas and display the first few records.
2. Perform exploratory data visualization by plotting a scatter plot between House Size and House Price. Briefly comment on the relationship between the variables.
3. Split the dataset into training and testing sets using an 80:20 ratio.
4. Build a Simple Linear Regression model using the training data.
5. Display the learned model parameters:
    - Regression Constant (θ_0)

- Regression Coefficient (θ_1)

6. Predict the house prices for the test dataset and compare the predicted values with the actual values.
7. Predict the house price for the following house sizes:
   - 1500 sq.ft.
   - 1700 sq.ft.
   - 2500 sq.ft.
8. Plot the regression line along with the original data points and interpret the obtained graph.
9. Evaluate the performance of the model using the following metrics:
   - Mean Absolute Error (MAE)
   - Mean Squared Error (MSE)
   - Root Mean Squared Error (RMSE)
   - Coefficient of Determination (R^2 Score)
10. Plot the Actual vs. Predicted values and briefly comment on the model's prediction accuracy.

September 2, 2026

Lab Task 1: Perceptron for Binary Classification Without Using Built-in Libraries

Objective:

Implement the Perceptron learning algorithm from scratch for binary classification.

Tasks:

1. Load a binary classification dataset.
2. Perform the necessary preprocessing.
3. Initialize the weights and bias.
4. Implement the weighted sum and step activation function.
5. Implement the Perceptron weight update rule.
6. Train the Perceptron for a specified number of epochs.
7. Predict the class labels for the test samples.
8. Compare the predicted labels with the actual labels.
9. Analyze the effect of different learning rates and numbers of epochs.

Suggested Dataset:

Iris dataset by selecting any two classes.

Lab Task 2: Perceptron for Binary Classification Using a Built-in Library

Objective:

Apply a built-in Perceptron classifier for binary classification and compare its performance with the implementation developed from scratch.

Tasks:

1. Load a binary classification dataset.
2. Perform the necessary preprocessing.

3. Split the dataset into training and testing sets.
4. Train a Perceptron classifier using a machine learning library.
5. Predict the class labels for the test samples.
6. Compare the predicted labels with the actual labels.
7. Compare the results with the Perceptron implementation developed from scratch.

Suggested Library:
Scikit-learn.

Lab Task 3: Logistic Regression for Binary Classification Using a Built-in Library

Objective:
Build and apply a Logistic Regression model for binary classification.

Tasks:

1. Load a binary classification dataset.
2. Separate the input features and target labels.
3. Perform the necessary preprocessing.
4. Split the dataset into training and testing sets.
5. Train a Logistic Regression model using a machine learning library.
6. Predict class probabilities and class labels.
7. Compare the predicted labels with the actual labels.
8. Analyze the effect of different classification thresholds on the predicted labels.

Suggested Dataset:
Breast Cancer Wisconsin dataset.

Lab Task 4: Logistic Regression from Scratch

Objective:
Implement Logistic Regression from scratch using gradient descent and compare its results with a library implementation.

Tasks:

1. Load a binary classification dataset.
2. Perform the necessary preprocessing.

3. Initialize the weights and bias.
4. Implement the sigmoid activation function.
5. Compute the predicted probabilities.
6. Calculate the prediction error.
7. Update the weights and bias using gradient descent.
8. Repeat the training process for a specified number of iterations.
9. Predict class labels for the test samples.
10. Compare the results with a built-in Logistic Regression model.

Multilayer Perceptron for Breast Cancer Classification

Aim

To implement a Multilayer Perceptron (MLP) classifier for breast cancer classification using:

1. a machine-learning library, and
2. a from-scratch implementation using NumPy, forward propagation, and backpropagation.

Dataset

Use the Breast Cancer Wisconsin Diagnostic Dataset available through `sklearn.datasets`.

The dataset contains:

- Number of samples: 569
- Number of features: 30
- Number of classes: 2
- Target classes: Malignant and Benign

Part A: MLP Using a Machine-Learning Library

Task

Develop an MLP classifier to predict whether a breast tumor is malignant or benign.

Students must perform the following steps:

1. Load the Breast Cancer dataset.
2. Separate the features and target variable.
3. Perform appropriate feature scaling.
4. Divide the dataset into training and testing sets.
5. Construct an MLP classifier containing:
    - Input layer
    - One or more hidden layers
    - Output layer
6. Train the MLP using the training data.
7. Predict the classes of the test data.
8. Evaluate the trained model.
9. Display the classification results.

Experimental Study

Students must investigate the effect of changing the following parameters:

No.	Parameter to Investigate
1	Number of hidden layers
2	Number of neurons in each hidden layer
3	Learning rate
4	Number of training iterations/epochs
5	Activation function

Table 1: MLP parameters for experimental study

Students should change one parameter at a time while keeping the other parameters fixed and observe the effect on model performance.

Questions

1. What happens to the model when the number of neurons in the hidden layer is increased?
2. What happens when the learning rate is very small?
3. What happens when the learning rate is very large?
4. Compare an MLP with one hidden layer and an MLP with two hidden layers.
5. Why is a nonlinear activation function required in the hidden layers?
6. What happens if all the layers use a linear activation function?
7. Why can increasing the number of hidden neurons sometimes lead to overfitting?

Part B: MLP From Scratch

Task

Implement an MLP from scratch for the same Breast Cancer Wisconsin Diagnostic Dataset.

The implementation must not use an existing MLP or neural-network implementation from a machine-learning library.

Students may use *NumPy* for matrix operations and numerical computations.

The implementation must include the following components:

1. Weight initialization
2. Bias initialization
3. Forward propagation

4. Activation function
5. Loss/error calculation
6. Backpropagation
7. Weight update
8. Bias update
9. Iterative training
10. Prediction